map-charting-student-math-misunderstandings

1位 Solution

Rank	1
Team	tascj
Author	tascj
Topic ID	612268
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/map-charting-student-math-misunderstandings/discussion/612268
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

Solution Writeup

この興味深いチャレンジを開催してくださったコンペのホストとKaggleに感謝いたします。 NLPコンペで初めて1位を獲得でき、とても嬉しいです！以下が私のsolution writeupです。

1. データ

重複を削除し、35,960サンプルになりました。"Category" で層化した5-foldの交差検証splitを作成しました。

2. モデリング

この問題を suffix classification タスクとしてモデル化しました。

同じcontext（prefix）が与えられたとき、候補の集合から正しいsuffixを予測するようにモデルを学習させます。

Contextは次のような形式です。

```
<|imChoices}
 {Answer}
 {MisconceptionCandidates}
 {MCend|>
<|im
```

Suffixは提出形式で表します。

```
:<|im_end|>
```

各 `QuestionId` には、その問題で起こりうるmisconceptionに基づいて、8個、10個、または12個のsuffix候補があります。

`[prefix ++ suffix0, prefix ++ suffix1, ...]` の最終トークンの特徴量を取り出して nn.Linear(hidden_size, 1) に入力してlogitを得て、cross-entropy lossを計算します。 実際には、FlexAttentionのカスタムattention maskを使い、入力を prefix 共有形式 `prefix ++ suffix0 ++ suffix1 ++ ...` に構成しました。

```
():
    causal = q_idx >= kv_idx
    is_prefix = suffix_ids[kv_idx] == -
    same_suffix = (suffix_ids[q_idx] == suffix_ids[kv_idx])
    same_doc = doc_ids[q_idx] == doc_ids[kv_idx]
     causal & (same_suffix | is_prefix) & same_doc
```

3. 学習

今年前半の `WSDM Cup` コンペに取り組んでいた際、最大32Bのモデルを単一のA100 80G GPUで効率的にfull-parameter学習できるように offload_adam を実装しました。そのため、このコンペではすべての実験を単一のA100 80GまたはRTX Pro 6000 Blackwellで実行できました。ほとんどの学習は同じハイパーパラメータ `epoch=1, batch_size=32, learning_rate=1e-5` で行いました。

おそらくラベルノイズ（主に `Neither`）のため、validationスコアはseedによって大きく変動しました。安定したvalidationスコアを得るには、異なるseedによる複数回の学習のアンサンブルを使う必要がありました。

`Qwen3-8B` で5-fold × 5-seedsの学習を行いました。3 seedのアンサンブルで安定するようだったので、以降の実験では最も難しいfoldと3回の学習のアンサンブルを使いました。この手法は新しいものではありません。"Feedback Prize - Predicting Effective Arguments" の 1st place solution も、validationに3-seedアンサンブルを使っていました。

主なValidation結果（seedアンサンブル [seed1, seed2, seed3]）

model	loss	MAP@3
deepseek-ai/DeepSeek-R1-Distill-Qwen-7B	0.2716 [0.2809, 0.2871, 0.2835]	0.9444 [0.9426, 0.9428, 0.9421]
Qwen/Qwen3-8B	0.2677 [0.2813, 0.2777, 0.2756]	0.9455 [0.9433, 0.9450, 0.9437]
zai-org/GLM-Z1-9B-0414	0.2627 [0.2783, 0.2762, 0.2761]	0.9469 [0.9455, 0.9433, 0.9452]
deepseek-ai/DeepSeek-R1-Distill-Qwen-14B	0.2621 [0.2782, 0.2698, 0.2738]	0.9464 [0.9421, 0.9443, 0.9450]
Qwen/Qwen3-14B	0.2614 [0.2707, 0.2744, 0.2695]	0.9477 [0.9442, 0.9451, 0.9461]
Qwen/Qwen3-32B	0.2589 [0.2687, 0.2718, 0.2700]	0.9484 [0.9465, 0.9451, 0.9454]
zai-org/GLM-Z1-32B-0414	0.2560 [0.2713, 0.2681, 0.2682]	0.9480 [0.9450, 0.9463, 0.9462]
Qwen/Qwen3-32B+zai-org/GLM-Z1-32B-0414	0.2530	0.9496

実験結果から、次のことがはっきりと示されました。

1. 単一seedのvalidationスコアを信じるべきではない
2. モデルが大きいほど性能が良い
3. MAP@3よりもlossを信じた方が良いかもしれない

multi-foldアンサンブルとmulti-seedアンサンブルも比較しました。multi-seedアンサンブルの方がはるかに良好でした。そのため最終提出では、`Qwen/Qwen3-32B` と `zai-org/GLM-Z1-32B-0414` を、それぞれ3つの異なるseedと少しずつ異なるデータ形式で、全データを使って学習しました。

補助SFT Lossの実験

コンペの最後の数日間に、`Qwen/Qwen3-235B-A22B-Thinking-2507-FP8` を使ってラベルの短い根拠（justification）を生成し、生成した内容に対する補助SFT lossでモデルを学習する実験を行いました。

生成した根拠の例:

```
True_Correct:NA
The categorization
```

この手法は `deepseek-ai/DeepSeek-R1-Distill-Qwen-7B` ではいくらか改善を示しましたが、`Qwen/Qwen3-8B` ではほとんど効果がありませんでした。アンサンブルには含めましたが、寄与はわずかだと見積もっています。いずれにせよ、興味深い試みでした。

4. 推論

推論には、計算とメモリという2つの主な課題があります。

計算

短いシーケンスのprefillという負荷では、主なボトルネックは `nn.Linear` 層の行列積です。T4はホワイトペーパー上ではFP16で65TFLOPSを謳っていますが、実際には不安定な20TFLOPSしか出せませんでした。

LMDeployのW8A8 INT8カーネルは、安定して40+TFLOPSを達成します。W8A8 INT8推論を可能にするため、SmoothQuant(alpha=0.75)でモデルを量子化しました。validationでのアンサンブル性能は、量子化していないモデルとほぼ同じです。

メモリ

単一のT4 GPUで32Bモデルを推論するため、layer-wise推論を使いました。

1. GPU上には2つのtransformer層だけを初期化して保持する
2. `現在の層の実行` と `次の層の状態をディスクから読み込むこと` をオーバーラップさせる

GPUを常に稼働させておくため、各forwardで640サンプル（16サンプルのmicro batch × 40）を使いました。

T4×2で16,000サンプルの推論を終えるのに約65分かかります。問題文のcontextのprefix-caching/sharingを実装すれば推論時間をさらに短縮できた（トークンの半分以上をキャッシュできた）はずですが、実装しませんでした。

Kaggle Notebook環境についてのメモ

Kaggle notebook環境のストレージにはいくつか悩まされました。

1. `/kaggle/input` - ローカルストレージではなく、アクセスが非常に遅い
2. `/tmp/` - 容量制限のあるローカルのCopy-on-Writeストレージで、**ファイルを本当に削除することはできない**
3. `/kaggle/working` - 通常のローカルストレージで、ファイルを削除して空き容量を確保できる

当初は層のcheckpointを `/tmp` に保存しており、容量オーバーでクラッシュしました。デバッグの後、（アップロードした6モデルのうち）4モデルのアンサンブルを提出する時間しか残りませんでした。それでも結果はかなり良好でした。

5. まとめ

私にとって、このコンペで最も重要だったのは、信頼できるvalidationスコアを得る正しい方法を見つけることでした。単一seedのvalidationスコアは非常に不安定で、誤解を招くものでした。multi-seedアンサンブルによって安定した信頼できる検証指標が得られ、それによって効果的なモデルのチューニングと選択が可能になりました。

Submission notebook Training

補足Q&A

AI-Cat — 質問

素晴らしい成果ですね。また、見事なsolutionの概要を少なくとも簡潔に共有してくださりありがとうございます。MAP@3という指標がLOSSより劣るという説得力のある結論に、何がきっかけで至ったのかが分かりません。サーブは見当たりません。カウントされるのは1回のサーブだけで、それはもうクイーンです。悪い指標なのか、モデルが過学習しているのかを、どうやって見分けるのでしょうか。この場合、最も注目すべき点は何でしょうか。

@kaggleqrdl — 質問

お見事です。この優勝で、あなたはコンペで新たな世界1位のkagglerになったと思います。

あなたのCVの手法が勝敗を決めた、と言ってよいでしょうか。今後の参考にぜひ取っておきたいと思います。

WOOSUNG YOON — 質問

詳細なsolutionをありがとうございます。とても参考になりました。

16サンプルのmicro-batch × 40について質問です。 この設定は、I/Oのボトルネックに対処するためのT4 GPU上のlayer-wise offloadingに特有のものですか、 それとも一般的な手法ですか？これを実装するコードも共有していただけますか？

正確に言うと、 GitHubに実装されている次の機能がこの仕組みに関係していそうです。 これがどのように動作するのか、言葉で説明していただけますか？

LX — 質問

```
I used the most fold ensemble of  runs for further experiments.
```

ここで言う「最も難しいfold」とは、5-fold交差検証の5つのfoldのうち1つをvalidationデータとして選び、3つの学習seedを使ってこの20%のvalidationセットのスコアを検証する、という意味でしょうか？

validationの後、テストセットで推論する際は、全データで学習し直すのでしょうか、それとも既に学習済みのモデルをそのまま使うのでしょうか？

tascj — 回答

validationスコアが最も低いfoldを、固定のvalidationセットとして選びました。各実験では、残り80%のデータで3つのseedを学習しました。

最終提出では、全データでモデルを学習し直しました。

tascj — 回答

これは慎重にはテストしていません。私の入力形式では、640サンプルあれば層の状態をディスクから読み込む時間を隠すのに十分でした。Kaggleのストレージは非常に遅いです。ローカルのNVMeストレージなら32サンプルで十分でしたが、ローカルのGPUの方がずっと高速です。 micro-batchに分けるのはOOMを防ぐためです。

submission notebookは公開しました。

tascj — 回答

他のチームのsolutionはまだ詳しく確認していませんが、多くのチームが2〜3 epoch学習していることに気づいたので、状況は多少異なるかもしれません。

tascj — 回答

これが厳密な結論ではないことには同意します。表のデータだけでは、この点を本当に証明しているとは言えません。

Psi — 質問

smooth quantとLMDeployを使って複数の大きなモデルをアンサンブルし、文字どおり最後の最後に提出したことには大きな敬意を表します。すべてが問題なく走り切るとどれくらい自信があったのか気になります :)

tascj — 回答

推論部分（w8a8量子化、layer-wise）はWSDM Cupコンペの際に開発したもので、既に検証済みでした。 当時、広範なテストを行いました。このタスクではサンプルのシーケンス長が非常に短いので、TFLOPSから推論時間を線形に直接見積もる方法で問題なく機能するはずです。

Bilzard — 質問

@tascj0 1回の提出だけでの1位、おめでとうございます。

> この問題を suffix classification タスクとしてモデル化しました。 同じcontext（prefix）が与えられたとき、候補の集合から正しいsuffixを予測するようにモデルを学習させます。

一見すると、suffix予測タスクはCAUSAL_LMやSEQ_CLSタスクに似ています。主な違いは、ラベルの表現が出力ではなく入力として与えられる点です（私の理解が正しければ）。

このコンペにおいて、CAUSAL_LMやSEQ_CLSタスクに対するsuffix予測の利点は何だとお考えですか？タスク設計そのものがこれらのタスクより優れているのでしょうか？

tascj — 回答

いくつかの異なるモデリングのパラダイムを試しましたが、それらの間に有意な性能差は見られませんでした。suffix classificationの手法を選んだのは、問題を定式化する最も自然な方法だと考えたからです。ラベル候補を入力として明示的にエンコードでき、候補の順序にも依存しません。また、総トークン数も妥当な量になりました。